

Question 2: Compare Permissions on Regular Files and Directories

Introduction

Linux uses a permission system to control access to files and directories. The three basic permissions are read (r), write (w), and execute (x). These permissions can be assigned separately to the owner, group, and other users.

Although the same permission symbols are used for both files and directories, their meaning is different. For a regular file, read permission allows the contents of the file to be viewed, write permission allows the contents to be modified, and execute permission allows the file to be executed when it is a valid executable program or script.

For a directory, read permission allows the names of files inside the directory to be listed, write permission allows directory entries to be created or removed when the required execute permission is also available, and execute permission allows the user to enter the directory and access its contents.

This experiment compares the behavior of permissions on a regular file and a directory using the `chmod` command. The experiment is performed as a normal Linux user without using `sudo`.

Objective

The main objectives of this experiment are:

- To understand Linux file and directory permissions.
- To study the difference between file and directory permissions.
- To use the `chmod` command to modify permissions.
- To observe the effect of read, write, and execute permissions.
- To understand why directory permissions behave differently from file permissions.

Creating the Required Files and Directory

First, a regular file named `sample.txt` and a directory named `sample_dir` were created.

```
touch sample.txt
mkdir sample_dir
touch sample_dir/test.txt
```

The contents of the directory can be checked using:

```
ls -l
ls -l sample_dir
```

A small amount of text can be added to the regular file for testing:

```
echo "Linux permission testing" > sample.txt
```

The directory now contains at least one file:

```
sample_dir/  
  test.txt
```

Understanding chmod

The chmod command is used to change the permissions of a file or directory.

For example:

```
chmod 400 sample.txt
```

The three digits represent permissions for:

Digit	Permission
4	Read (r)
2	Write (w)
1	Execute (x)

Therefore:

```
400 = r--  
200 = -w-  
100 = --x  
500 = r-x  
600 = rw-  
700 = rwx
```

Permission Testing and Observations

Perm.	Regular File Test	Observation	Directory Test	Observation
400	Try to read and modify the file.	File can be read, but modification is denied because write permission is absent.	Try to list and enter the directory.	Names may be read, but entering/accessing the directory is denied because execute permission is absent.
200	Try to read and modify the file.	Reading is denied. Modification of the file contents is allowed in principle because write permission is present.	Try to list, enter, and create a file.	Without read and execute permissions, the directory cannot be properly listed, entered, or used to create files.
100	Try to read and execute the file.	Reading is denied. Execution is permitted only if the file is a valid executable or executable script.	Try to list and enter the directory.	The directory can be entered, but its contents cannot be listed because read permission is absent.
500	Try to read and modify the file.	File can be read but cannot be modified because write permission is absent.	Try to list and enter the directory.	Directory contents can be listed and the directory can be entered because both read and execute are present.
600	Try to read and modify the file.	Both reading and modification are allowed. Execution is not allowed.	Try to list, enter, and create a file.	Directory names can be listed, but entering or creating files is not possible because execute permission is absent.
700	Try to read, modify, and execute the file.	Reading, modification, and execution are permitted, provided the file is executable.	Try to list, enter, create, and delete files.	Full access is available to the owner. The owner can list, enter, create, modify, and delete directory entries.

Commands Used During the Experiment

The following commands can be used to test each permission setting.

Testing 400 Permission

```
chmod 400 sample.txt
cat sample.txt
echo "test" >> sample.txt
```

```
chmod 400 sample_dir
ls sample_dir
cd sample_dir
```

With 400 on the file, the contents can be read but the file cannot be modified. On the directory, read permission alone is not enough to access the directory properly because execute permission is missing.

Testing 200 Permission

```
chmod 200 sample.txt
cat sample.txt
echo "test" >> sample.txt
```

```
chmod 200 sample_dir
ls sample_dir
cd sample_dir
```

A regular file with only write permission cannot be read, but its contents can be modified. For a directory, write permission alone is not sufficient for normal file creation or deletion because execute permission is also required.

Testing 100 Permission

```
chmod 100 sample.txt
cat sample.txt
```

```
chmod 100 sample_dir
ls sample_dir
cd sample_dir
```

For a regular file, execute permission does not automatically make the file readable. If the file is not a valid executable program or script, attempting to execute it will fail.

For a directory, execute permission allows the user to enter the directory and access known files, but the directory contents cannot be listed without read permission.

Testing 500 Permission

```
chmod 500 sample.txt
cat sample.txt
echo "test" >> sample.txt
```

```
chmod 500 sample_dir
ls sample_dir
cd sample_dir
```

The file can be read but cannot be modified. The directory can be listed and entered because both read and execute permissions are available. However, files cannot be created or deleted because write permission is missing.

Testing 600 Permission

```
chmod 600 sample.txt
cat sample.txt
echo "test" >> sample.txt
```

```
chmod 600 sample_dir
ls sample_dir
cd sample_dir
```

For the regular file, both read and write operations are possible. For the directory, the names can be read, but the user cannot enter or properly access its contents because execute permission is missing.

Testing 700 Permission

```
chmod 700 sample.txt
cat sample.txt
echo "test" >> sample.txt
```

```
chmod 700 sample_dir
ls sample_dir
cd sample_dir
touch sample_dir/newfile.txt
rm sample_dir/newfile.txt
```

With 700 permission, the owner has read, write, and execute permissions. A directory with 700 permissions provides full access to the owner, including listing, entering, creating, modifying, and deleting files.

Questions and Answers

1. What does read (r) permission allow you to do with a regular file?

Read permission on a regular file allows the user to view or read the contents of that file. For example, if `sample.txt` has read permission, the `cat` command can be used to display its contents.

```
chmod 400 sample.txt
cat sample.txt
```

However, read permission does not allow the contents to be changed. If write permission is not present, an attempt to modify the file will normally result in a permission denied error.

Therefore, for a regular file, read permission mainly controls the ability to view its contents.

2. What does read (r) permission allow you to do with a directory?

Read permission on a directory allows the user to view the names of the files and subdirectories stored inside it. For example, the command:

```
ls sample_dir
```

can display directory entries when the appropriate read permission is available.

However, read permission alone does not allow the user to enter the directory or access files inside it. Execute permission is normally required along with read permission for useful access to directory contents.

Thus, directory read permission means the ability to see the directory's contents, not necessarily the ability to access those contents.

3. What does write (w) permission mean for a regular file?

Write permission on a regular file allows the contents of the file to be changed. A user with write permission can add new information, remove existing information, or replace the contents of the file.

For example:

```
chmod 200 sample.txt
echo "New content" >> sample.txt
```

The important point is that write permission controls the contents of the file. It does not by itself mean that the user can delete the file. Deleting a file is mainly controlled by the permissions of the directory containing the file.

4. What does write (w) permission mean for a directory?

Write permission on a directory allows changes to the directory entries. This includes creating new files and removing existing files, provided the required execute permission is also available.

For example, a directory with `wx` permissions can be used to create or delete entries even when the user cannot list the directory contents.

This is an important difference between files and directories. Write permission on a directory does not mean that the user can directly modify the contents of every file inside it.

5. What does execute (x) permission mean for a regular file?

Execute permission on a regular file allows the operating system to attempt to run the file as a program. However, the file must actually contain a valid executable program or an executable script.

For example:

```
chmod 700 program.sh
./program.sh
```

If a normal text file is given execute permission, it does not automatically become a useful executable program. Therefore, execute permission controls whether the file can be executed, but it does not convert the file into a program.

6. Why is execute (x) permission important for accessing a directory?

Execute permission on a directory means that the user is allowed to enter the directory and access entries within it, provided other required permissions are present.

For example:

```
cd sample_dir
```

requires execute permission on the directory.

Execute permission is therefore sometimes described as the “search” permission for directories. Without it, a user may know the name of a file inside the directory but still be unable to access that file.

For normal directory usage, read and execute permissions are commonly used together:

```
r-x
```

Read allows the directory entries to be listed, while execute allows those entries to be accessed.

7. Which permission combination allows the owner to fully access a regular file?

The permission combination is:

700 = `rwX-----`

This gives the owner read, write, and execute permissions while giving no permissions to the group or other users.

For a regular file:

- **r** allows reading the contents.
- **w** allows modifying the contents.
- **x** allows execution when the file is a valid executable.

Therefore, 700 provides complete permission from the Linux permission perspective for the owner.

8. Which permission combination allows the owner to fully access a directory?

The permission combination is:

700 = `rwX-----`

For a directory, these permissions mean:

- **r** allows the owner to list directory entries.
- **w** allows directory entries to be created or removed.
- **x** allows the owner to enter and access the directory.

Therefore, 700 gives the owner full control over the directory while keeping the directory inaccessible to the group and other users.

9. Why do the same r, w, and x permissions behave differently for a regular file and a directory?

The permissions behave differently because a regular file and a directory represent different types of objects in the Linux file system.

For a regular file, the permissions mainly control operations on the file's contents. Read means viewing the contents, write means modifying the contents, and execute means running the file when it is executable.

For a directory, the permissions control operations on the directory itself and its entries. Read allows the names of entries to be listed, write allows entries to be created or removed, and execute allows the directory to be entered and searched.

For example, a directory with read permission but without execute permission may allow the names of files to be seen, but the user may not be able to open or access those files through that directory.

Similarly, write permission on a directory does not mean that the user can modify the contents of every file inside it. It means that the user can make changes to the directory's entries when execute permission is also available.

Therefore, the meaning of a permission depends on the type of file-system object to which it is applied.

Summary of File and Directory Permissions

Permission	Regular File	Directory
Read (r)	Read the contents of the file.	List the names of entries in the directory.
Write (w)	Modify the contents of the file.	Create or remove directory entries when execute permission is also available.
Execute (x)	Execute the file if it is a valid executable or script.	Enter, search, and access entries in the directory.

Conclusion

This experiment shows that Linux permissions are not simply a set of permissions that have the same meaning everywhere. The effect of a permission depends on whether it is applied to a regular file or a directory.

For regular files, read, write, and execute permissions mainly control access to the file's contents and execution. For directories, read, write, and execute permissions control listing, modification of directory entries, and access to the directory.

The most important observation from the experiment is that execute permission has a different purpose for directories. It acts as a search or access permission and is necessary for entering the directory and accessing its contents. Similarly, deleting a file depends mainly on the permissions of its parent directory rather than the write permission of the file itself.

Understanding these differences is important when working with Linux systems because incorrect permissions can either prevent legitimate access or expose files and directories to unauthorized users. The `chmod` command provides a simple way to control these permissions according to the requirements of the system.